

Основи роботи з фреймворком Spring Boot

Тема 2: REST, JSON, Валідація

Бакалаврська сертифікатна програма
«Розробка бізнес-застосунків на Java»

Зміст лекції

1. Архітектура REST та HTTP
2. Серверний стек та продуктивність
3. Розробка контролерів у Spring Web
4. Робота з JSON та Jackson
5. Обробка помилок та стандартизація
6. Валідація даних (Jakarta Validation)
7. Клієнтська взаємодія та стандарти

1. Архітектурний контекст та еволюція розподілених систем

Розвиток інтеграції систем пройшов кілька ключових етапів:

- **CORBA та RPC:** Жорстка бінарна прив'язка до мов та мережевої інфраструктури, складність налаштування.
- **SOAP (Simple Object Access Protocol):** Важковагові XML-повідомлення, складні схеми XSD, надмірне споживання ресурсів.
- **REST (Representational State Transfer):** Використання нативних стандартів HTTP, легковагові формати (JSON) та слабка зв'язність.

Підхід **API-First** розглядає контракт взаємодії (OpenAPI) як первинний артефакт, що дозволяє паралельну розробку фронтенду та бекенду.

Проблема взаємодії гетерогенних систем

Як спроектувати взаємодію клієнтів (веб, мобільні пристрої, IoT) та бекенд-сервісів, щоб архітектура залишалася масштабованою та не залежала від мов програмування?

Які обмеження протоколу зв'язку гарантують простоту кешування та незалежність компонентів?

Відповідь: Принципи та обмеження REST

У 2000 році Рой Філдінг сформулював 6 фундаментальних архітектурних обмежень REST:

1. **Client-Server:** Чітке розділення відповідальності інтерфейсу користувача та бізнес-логіки.
2. **Stateless:** Кожен запит містить усю необхідну інформацію для його обробки сервером.
3. **Cacheable:** Відповіді явно позначаються як кешовані або некешовані.

Архітектурні обмеження REST

- 4. **Uniform Interface:** Уніфікований інтерфейс ідентифікації та маніпулювання ресурсами.
- 5. **Layered System:** Можливість використання проміжних проксі, шлюзів та балансувальників.
- 6. **Code on Demand:** Опціональна передача виконуваного коду на клієнт.

Переваги Stateless-архітектури

Відсутність сесійного стану на сервері забезпечує ключові характеристики:

- **Горизонтальне масштабування:** Запити користувача можуть оброблятися будь-яким екземпляром без синхронізації пам'яті.
- **Відмовостійкість:** Падіння окремого вузла кластера не призводить до втрати контексту сесії користувача.
- **Токенна автентифікація:** Використання підписаних токенів (JWT) передає контекст авторизації у кожному HTTP-запиті.
- **Простота моніторингу:** Кожен запит є автономним та може бути ізольовано проаналізований у системі трейсингу.

Проектний підхід до іменування URL

URL ідентифікує ресурси, а не дії чи операції над ними:

- **Використання іменників у множині:** `/api/v1/orders` замість `/api/v1/getOrders` або `/api/v1/createOrder`.
- **Ієрархічні зв'язки сутностей:** `/api/v1/users/{userId}/orders/{orderId}` для залежних ресурсів.
- **Пагінація, фільтрація та сортування:** Використання Query Parameters замість окремих ендпоінтів:

```
GET /api/v1/books?genre=tech&page=0&size=20&sort=title,asc
```

- **Версіонування:** Явне зазначення версії у шляху (`/api/v1/`) або через HTTP-заголовки (`Accept`).

Семантика HTTP-методів та ідемпотентність

Методи протоколу HTTP мають чітко визначену специфікацію:

Метод	Призначення	Безпечний (Safe)	Ідемпотентний
GET	Отримання представлення ресурсу	Так	Так
POST	Створення нового ресурсу	Ні	Ні
PUT	Повне оновлення ресурсу (заміна)	Ні	Так
PATCH	Часткове оновлення полів ресурсу	Ні	Ні
DELETE	Видалення існуючого ресурсу	Ні	Так

Повторний виклик ідемпотентного методу призводить до того ж стану системи, що й одноразовий.

Стандартизація кодів відповідей HTTP

Використання стандартних статус-кодів формує передбачуваний контракт для клієнта:

- **2xx (Успішна обробка):** `200 OK` (успіх з тілом), `201 Created` (створено новий ресурс), `204 No Content` (успіх без тіла, наприклад при DELETE).
- **3xx (Перенаправлення):** `304 Not Modified` (ефективне кешування з валідацією ETag).
- **4xx (Помилки клієнта):** `400 Bad Request` (помилка синтаксису чи валідації), `401 Unauthorized` (відсутня автентифікація), `403 Forbidden` (недостатньо прав), `404 Not Found`, `409 Conflict`.
- **5xx (Помилки сервера):** `500 Internal Server Error`, `503 Service Unavailable`.

2. Веб-сервери та продуктивність у Spring Boot

Spring Boot підтримує роботу з різними вбудованими сервлет-контейнерами:

- **Apache Tomcat:** Вбудований за замовчуванням контейнер, перевірений часом та максимально стабільний.
- **Undertow (Red Hat):** Високопродуктивний сервер на базі неблокуючого I/O (XNIO), має менше споживання пам'яті під навантаженням.
- **Eclipse Jetty:** Легковажний контейнер, популярний у хмарних та мікросервісних середовищах.

Заміна сервера здійснюється виключенням стандартного стартера у файлі збірки та підключенням `spring-boot-starter-undertow`.

3. Анатомія REST-контролера у Spring Boot

Анотація `@RestController` є спеціалізованою композицією двох анотацій:

1. **@Controller**: Реєструє клас як компонент обробки запитів у Spring MVC.
2. **@ResponseBody**: Автоматично серіалізує повернуте методом значення у тіло HTTP-відповіді (за замовчуванням у JSON).

```
@RestController
@RequestMapping("/api/v1/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }
}
```

Обробка параметрів: @PathVariable

Отримання динамічних змінних безпосередньо зі структури шляху URI:

```
@GetMapping("/{id}")  
public UserResponse getById(@PathVariable("id") UUID id) {  
    return userService.findById(id);  
}
```

- **Призначення:** Ідентифікація конкретного ресурсу або зв'язку в ієрархії REST.
- **Типізація:** Spring автоматично виконує приведення типу рядка до `UUID`, `Long` тощо.
- **Іменування:** Якщо назва параметра збігається зі змінною у шаблоні, атрибут можна опустити.

Обробка параметрів: @RequestParam

Отримання параметрів рядка запиту (Query Parameters) з URI:

```
@GetMapping
public List<UserResponse> list(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size) {
    return userService.findAll(page, size);
}
```

- **Призначення:** Фільтрація, пагінація, сортування та передача параметрів.
- **Необов'язковість:** Атрибут `required = false` дозволяє обробляти опціональні дані.
- **Значення за замовчуванням:** `defaultValue` запобігає передачі `null`.

Обробка параметрів: @RequestHeader

Отримання службових HTTP-заголовків та контекстних метаданих:

```
@GetMapping("/info")
public ClientInfo info(@RequestHeader("User-Agent") String agent,
                      @RequestHeader(name = "X-Trace-Id", required = false) String traceId) {
    return new ClientInfo(agent, traceId);
}
```

- **Призначення:** Читання метаданих безпосередньо з заголовків HTTP-запиту.
- **Наскрізне трасування:** Кореляційні ідентифікатори (X-Trace-Id) для моніторингу.
- **Метадані:** Токени авторизації, локаль (Accept-Language), версія API.

Прийом даних через @RequestBody

Анотація `@RequestBody` вказує Spring десеріалізувати тіло HTTP-запиту в об'єкт Java за допомогою `HttpMessageConverter` :

```
@PostMapping
public ResponseEntity<Void> create(@RequestBody @Valid CreateUserRequest request) {
    UUID createdId = userService.create(request);
    return ResponseEntity.status(HttpStatus.CREATED).build();
}
```

- Вміст запиту розбирається відповідно до заголовка `Content-Type: application/json` .
- Якщо JSON має некоректний синтаксис, генерується виняток `HttpMessageNotReadableException` .

Формування відповідей: ResponseEntity

Об'єкт `ResponseEntity<T>` надає повний контроль над HTTP-відповіддю:

```
// Повернення статусу 200 OK з об'єктом у тілі
return ResponseEntity.ok(userDto);

// Повернення статусу 404 Not Found без тіла
return ResponseEntity.notFound().build();

// Встановлення кастомних заголовків та статусу
return ResponseEntity.status(HttpStatus.ACCEPTED)
    .header("X-Processing-Time", "42ms")
    .body(taskStatus);
```

Використання `ResponseEntity` дозволяє чітко дотримуватися специфікації REST у кожному сценарії.

Стандарт REST: Заголовок Location при створенні

При створенні нового ресурсу (POST) гарною практикою є повернення статусу 201 Created та заголовка Location з посиланням на новий ресурс:

```
@PostMapping
public ResponseEntity<Void> createUser(@RequestBody @Valid CreateUserRequest req) {
    UUID userId = userService.create(req);

    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(userId)
        .toUri();

    return ResponseEntity.created(location).build();
}
```

Клієнт отримує точну адресу новоствореного ресурсу без передачі надлишкового тіла відповіді.

Практика: CRUD-контролер на сучасній Java

Комбінація сучасних можливостей Java 25 та Spring Web:

```
@RestController
@RequestMapping("/api/v1/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<UserResponseDto> getById(@PathVariable UUID id) {
        return userService.findById(id)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

4. Обробка JSON за допомогою Jackson

Бібліотека Jackson є базовим інструментом серіалізації у Spring Boot:

- **Серіалізація:** Перетворення Java-об'єктів у текстовий формат JSON під час відправки клієнту.
- **Десеріалізація:** Парсинг рядка JSON та створення екземплярів класів Java під час обробки вхідного запиту.
- **ObjectMapper:** Центральний клас Jackson, який Spring налаштовує автоматично згідно з параметрами додатку.

```
Java Object (UserRecord) <==== ObjectMapper ====> JSON String {"name":"Ivan"}
```

Вразливість Mass Assignment у REST API

Що відбудеться, якщо зловмисник надішле у POST-запиті поле `"role": "ADMIN"`, якщо це поле не передбачено публічною формою реєстрації?

Як налаштувати парсер JSON так, щоб запобігти прихованому перезапису внутрішніх полів сутностей?

Відповідь: Безпечна конфігурація Jackson

У файлі `application.properties` слід налаштувати суворий режим обробки невідомих властивостей:

```
# Заборона невідомих полів у запиті (помилка 400 Bad Request)
spring.jackson.deserialization.fail-on-unknown-properties=true

# Виключення порожніх полів з відповіді для оптимізації трафіку
spring.jackson.default-property-inclusion=non_null

# Стандартизований формат дат ISO-8601 замість таймстемпів
spring.jackson.serialization.write-dates-as-timestamps=false
```

Ці параметри захищають систему від ін'єкції неавторизованих полів та покращують читабельність JSON.

Ключові анотації Jackson

Jackson дозволяє точно керувати процесом серіалізації на рівні полів DTO:

- **@JsonProperty**: Вказує власне ім'я поля в JSON
(`@JsonProperty("registered_at")`).
- **@JsonIgnore**: Повністю виключає чутливе поле із серіалізації (наприклад, хеш пароля).
- **@JsonFormat**: Задає шаблон форматування складних типів (дат, чисел):

```
@JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "yyyy-MM-dd'T'HH:mm:ss")  
Instant createdAt
```
- **@JsonTypeInfo**: Забезпечує коректну серіалізацію поліморфних структур та ієрархій класів.

Мутабельність DTO проти незмінності

Чи повинні об'єкти передачі даних (DTO) бути мутабельними та містити сетери після отримання з мережі?

Які ризики несе зміна стану DTO у різних шарах бізнес-логіки застосунку?

Відповідь: Records як стандарт DTO у Java 25

Використання бібліотеки Lombok (`@Data`) для DTO вважається застарілою практикою.

Переваги Java Records для REST

- **Незмінність (Immutable):** Усі компоненти запису є `final`, стан захищено від мутацій.
- **Лаконічність:** Автоматично генеруються канонічний конструктор, методи доступу, `equals()`, `hashCode()` та `toString()`.
- **Безшовна інтеграція:** Повна підтримка Jackson та нативна сумісність з GraalVM AOT без плагінів генерації байткоду.

Захист та інкапсуляція у Records

Записи дозволяють використовувати **компактні конструктори** для валідації та забезпечення глибокої незмінності колекцій:

```
public record CreateUserRequest(  
    String username,  
    String email,  
    List<String> roles  
) {  
    public CreateUserRequest {  
        if (username == null || username.isBlank()) {  
            throw new IllegalArgumentException("Ім'я користувача є обов'язковим.");  
        }  
        // Захист від мутацій колекції ззовні  
        roles = (roles != null) ? List.copyOf(roles) : List.of();  
    }  
}
```

5. Обробка помилок у REST API

Еволюція підходів до обробки винятків у Spring-застосунках:

1. Блоки `try-catch` у кожному контролері: Призводять до колосального дублювання коду та засмічення бізнес-логіки.
2. Анотація `@ExceptionHandler` у контролері: Дозволяє перехоплювати помилки лише в межах одного конкретного класу.
3. Глобальний перехоплювач `@RestControllerAdvice`: Аспектний підхід, що централізовано обробляє винятки для всіх контролерів застосунку.

Проблема витоку інформації через Stack Trace

Чому повернення стандартної сторінки помилки сервера зі слідом виконання (Stack Trace) та назвами таблиць БД є критичною вразливістю безпеки?

Яким має бути структурований формат відповіді про помилку для фронтенду?

Відповідь: Стандарт RFC 7807 Problem Details

Індустріальний стандарт RFC 7807 встановлює єдиний формат опису помилок у заголовку `application/problem+json`:

- **type:** URI-ідентифікатор типу помилки (посилання на документацію API).
- **title:** Короткий зрозумілий опис категорії помилки.
- **status:** Відповідний HTTP статус-код.
- **detail:** Конкретний опис причини виникнення помилки для даного запиту.
- **instance:** URI ресурсу, на якому сталася помилка.

Усі клієнти отримують передбачувану структуру помилки незалежно від ендпоінтів.

Налаштування ProblemDetail у Spring Boot 4.1

Spring Boot 4 має нативну підтримку стандарту RFC 7807.

Увімкнення стандарту для стандартних системних помилок у `application.properties`:

```
spring.mvc.problem-details.enabled=true
```

Клас `ProblemDetail` представляє тіло стандартизованої відповіді та дозволяє додавати кастомні метадані:

```
ProblemDetail problem = ProblemDetail.forStatusAndDetail(  
    HttpStatus.NOT_FOUND, "Користувача не знайдено");  
problem.setProperty("timestamp", Instant.now());
```

Реалізація @RestControllerAdvice

Створення глобального обробника бізнес-винятків застосунку:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ProblemDetail handleNotFound(ResourceNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Ресурс не знайдено");
        pd.setType(URI.create("https://api.example.com/errors/not-found"));
        return pd;
    }
}
```

6. Валідація вхідних даних (Jakarta Validation)

Ніколи не можна довіряти вхідним даним, що надходять від клієнта:

- **Hibernate Validator:** Еталонна реалізація специфікації Jakarta Bean Validation (JSR 380).
- **Підключення:** Стартер `spring-boot-starter-validation`.
- **Принцип роботи:** Валідація виконується автоматично перед передачею об'єкта в метод контролера за наявності анотації `@Valid`:

```
@PostMapping  
public ResponseEntity<Void> register(@RequestBody @Valid CreateUserRequest request) { ... }
```


Базові та просунуті анотації валідації

Jakarta Validation надає широкий набір готових анотацій:

- **@NotNull, @NotBlank, @NotEmpty:** Перевірка на відсутність null та порожніх рядків.
- **@Size(min = 3, max = 50):** Обмеження довжини рядка або розміру колекції.
- **@Min, @Max:** Перевірка числових діапазонів.
- **@Email:** Перевірка коректності формату електронної адреси.
- **@Past, @Future:** Перевірка часових рамок для дат.
- **@Pattern(regex = "^\\+380\\d{9}\$"):** Перевірка рядка регулярним виразом.

Проблема різних правил валідації для одного DTO

Як використовувати один DTO для операцій створення та оновлення сутності, якщо при створенні ID має бути порожнім, а при оновленні — обов'язковим?

Чи варто створювати окремі класи на кожну операцію, чи є спосіб розділити валідацію за контекстом?

Відповідь: Групи валідації (Validation Groups)

Групи валідації дозволяють застосовувати різні набори правил до одного DTO за допомогою інтерфейсів-маркерів:

```
public interface OnCreate {}
public interface OnUpdate {}

public record UserDto(
    @Null(groups = OnCreate.class)
    @NotNull(groups = OnUpdate.class)
    UUID id,

    @NotBlank(groups = {OnCreate.class, OnUpdate.class})
    String name
) {}
```

У контролері використовується анотація `@Validated(OnCreate.class)` замість `@Valid`.

Створення кастомних валідаторів

Для специфічної бізнес-перевірки створюється власна анотація та клас-валідатор:

```
@Target({ElementType.FIELD, ElementType.RECORD_COMPONENT})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = ComplexPasswordValidator.class)
public @interface ComplexPassword {
    String message() default "Пароль не відповідає вимогам безпеки";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

Клас валідатора імплементує інтерфейс `ConstraintValidator<ComplexPassword, String>` та підтримує впровадження Spring-бінів через Dependency Injection.

Обробка помилок валідації у ProblemDetail

При невдалій валідації `@Valid` генерує виняток `MethodArgumentNotValidException` :

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
    ProblemDetail pd = ProblemDetail.forStatusAndDetail(
        HttpStatus.BAD_REQUEST, "Помилка валідації вхідних даних");

    Map<String, String> errors = ex.getBindingResult().getFieldErrors().stream()
        .collect(Collectors.toMap(
            FieldError::getField,
            FieldError::getDefaultMessage,
            (first, second) -> first));

    pd.setProperty("errors", errors);
    return pd;
}
```

7. Декларативні HTTP-інтерфейси (HTTP Interfaces)

Spring пропонує сучасний підхід до взаємодії із зовнішніми REST API без написання реалізацій:

```
public interface UserClient {
    @GetExchange("/users/{id}")
    UserResponseDto getUserById(@PathVariable("id") UUID id);
}

// Конфігурація клієнта
RestClient restClient = RestClient.builder().baseUrl("https://api.service.com").build();
HttpServiceProxyFactory factory = HttpServiceProxyFactory
    .builderFor(RestClientAdapter.create(restClient)).build();
UserClient client = factory.createClient(UserClient.class);
```

Фреймворк самостійно генерує код клієнта на основі анотацій інтерфейсу.

Типові антипатерни у REST API (Anti-Patterns)

- **Дієслова у шляхах URL:** Використання `/api/createUser` замість `POST /api/users`.
- **Повернення 200 OK при помилках:** Приховування винятків усередині тіла успішної відповіді замість статусів 4xx/5xx.
- **Витік Stack Trace:** Передача деталей реалізації та внутрішніх винятків у відповідях клієнту.
- **Мутабельні DTO на базі Lombok:** Використання `@Data` замість компактних незмінних `record`.
- **Відсутність валідації на бекенді:** Сліпа довіра до коректності даних, надісланих з фронтенду.

Кращі індустріальні практики (Best Practices)

- **Stateless-дизайн:** Повна автономність кожного HTTP-запиту та використання токенів замість сесій сервера.
- **Стандарт RFC 7807:** Уніфікований формат `ProblemDetail` для передбачуваної обробки помилок.
- **Декларативна валідація:** Перевірка контрактів на вході через Jakarta Bean Validation.
- **Незмінність DTO:** Використання Java Records для гарантії безпеки стану та потокової безпеки.
- **Явні HTTP-статуси:** Коректне використання 201 Created, 204 No Content, 400 Bad Request, 404 Not Found.

Підсумки

Ключові висновки

1. **REST** базується на чітких обмеженнях, серед яких головними є **Stateless** та єдиний інтерфейс.
2. **Spring Boot 4.1** та **Virtual Threads** дозволяють обробляти високі I/O навантаження без переходу на реактивний стек.
3. **Java 25 Records** є індустріальним стандартом для створення надійних, незмінних DTO.
4. **RFC 7807 (ProblemDetail)** гарантує передбачуваний та безпечний формат передачі помилок клієнтам.
5. **Jakarta Bean Validation** забезпечує декларативну перевірку контрактів ще до виклику бізнес-логіки.

Індивідуальне завдання

Дедлайн: до кінця наступної доби

Завдання: Створити повноцінний REST-контролер для сутності власної предметної області з валідацією та збереженням у пам'яті.

(Результати завантажити на distedu до дедлайну)

Порядок виконання

1. Обрати сутність відповідно до власної предметної області (наприклад, `Book`, `User`, `Order`).
2. Описати структури даних DTO для операцій створення та читання.
3. Реалізувати логіку обробки запитів у контролері та збереження в колекції.

Вимоги до індивідуального завдання

1. **Сутності та DTO:** Повна незмінність моделей передачі даних з використанням Java Records (`record`).
2. **CRUD-операції:** Реалізація ендпоінтів GET, POST, PUT, DELETE у `@RestController` зі збереженням у колекції в пам'яті (`List` або `Map`).
3. **Декларативна валідація:** Використання анотацій `@NotBlank` , `@Size` , `@Email` або числових меж на полях вхідного DTO.
4. **Обробка винятків:** Реалізація `@RestControllerAdvice` для повернення помилок валідації у форматі `ProblemDetail` (RFC 7807).

Групове завдання

Дедлайн: 1 тиждень

Завдання: Спроектувати та реалізувати шар REST API для ключового бізнес-кейсу вашого командного проєкту.

Вимоги до групового завдання

1. **Контракти та іменування:** Чіткий опис REST-ендпоінтів для затвердженого бізнес-кейсу з дотриманням стандартів іменування ресурсів (іменники у множині, правильні HTTP-методи).
2. **Незмінність даних:** Реалізація DTO для вхідних та вихідних даних виключно на базі Java Records.
3. **Валідація та безпека:** Перевірка вхідних даних анотаціями Jakarta Validation та налаштування відхилення невідомих полів JSON.
4. **Єдиний формат помилок:** Обробка бізнес-винятків та помилок валідації через глобальний перехоплювач у форматі ProblemDetail (RFC 7807).
5. **Тестування:** Базове покриття контролерів тестами взаємодії.